Injecting Nested Objects Using XML Configuration

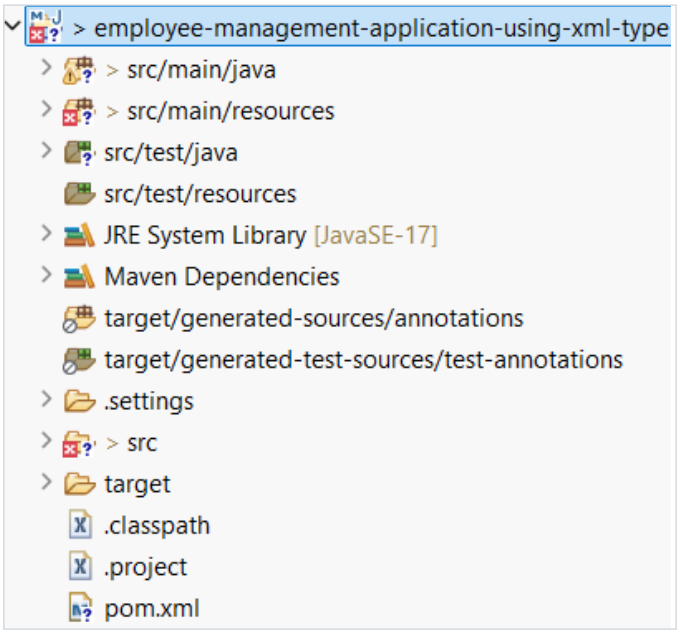
Spring Framework | XML-based bean configuration | ref attribute | Eclipse + Maven

Prerequisite: Create the Maven project

Same as the previous PDF (**Creating and Managing Spring Beans Using XML Configuration**). Only the values change:

Group Id	com.amazon.user
Artifact Id	employee-management-application-using-xml-type
Version	0.0.1-SNAPSHOT
Package	com.amazon.employee

A project will be created with the artifact name we have set, **employee-management-application-using-xml-type**, inside the Package Explorer in Eclipse (see SS1). The project uses **Java 17** (JavaSE-17 under JRE System Library).



SS1

Step 1: Add the Spring Context dependency

Open **pom.xml** and add the same **spring-context** dependency (version **6.2.19**) that we used in the previous PDF:

```
<!-- Source: https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.19</version>
    <scope>compile</scope>
</dependency>
```

Step 2: Understand the nested object

In the previous PDF, every field of **User** was a simple value (an `int` or a `String`). Here, an **Employee** *has an* **Address** and *has a* **Department**. These two fields are not primitives or Strings, they are **objects of our own classes**. So Spring must create the **Address** and **Department** objects first, and then hand them over to the **Employee**.

Class	Fields
Address	`String city, String state, String country`
Department	`int departmentId, String departmentName`
Employee	`int id, String name, Address address, Department department`

Step 3: Create the POJO classes

Create three classes inside **src/main/java**, in the package **com.amazon.employee**: **Address.java**, **Department.java** and **Employee.java**. Each class has only an **all-args constructor** (no default constructor, no setters), so Spring will use `<constructor-arg>` to create them.

Address.java

```
package com.amazon.employee;

public class Address {

    private String city;
    private String state;
    private String country;

    public Address(String city, String state, String country) {
        super();
        this.city = city;
        this.state = state;
        this.country = country;
    }

    @Override
    public String toString() {
        return "\n\tcity = " + city +
                "\n\tstate = " + state +
                "\n\tcountry=" + country;
    }

}
```

Department.java

```
package com.amazon.employee;

public class Department {

    private int departmentId;
    private String departmentName;

    public Department(int departmentId, String departmentName) {
        super();
        this.departmentId = departmentId;
        this.departmentName = departmentName;
    }

    @Override
    public String toString() {
        return departmentName;
    }
}
```

Employee.java

```
package com.amazon.employee;

public class Employee {

    private int id;
    private String name;

    private Address address;
    private Department department;

    public Employee(int id, String name, Address address, Department department) {
        super();
        this.id = id;
        this.name = name;
        this.address = address;
        this.department = department;
    }

    public void displayEmployeeInfo() {
        System.out.println(
            "Employee Details: "
            + "\nName: " + name
            + "\nAddress: " + address
            + "\nDepartment: " + department
        );
    }
}
```

Look at the last two fields of **Employee**: `Address address` and `Department department`. These are the nested objects. Printing them inside `displayEmployeeInfo()` calls their `toString()` methods automatically.

Step 4: Configure beans.xml

Create **beans.xml** inside **src/main/resources/** and paste the schema snippet, exactly as we did in Step 3 and Step 4 of the previous PDF. Then write the three beans below the line `<!-- bean definitions here -->`:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- bean definitions here -->

    <bean class="com.amazon.employee.Address" id="address">
        <constructor-arg value="BLR"/>
        <constructor-arg value="Karnataka"/>
        <constructor-arg value="India"/>
    </bean>

    <bean class="com.amazon.employee.Department" id="department">
        <constructor-arg value="501"/>
        <constructor-arg value="engineering"/>
    </bean>

    <bean class="com.amazon.employee.Employee" id="employee">
        <constructor-arg value="101"/>
        <constructor-arg value="Alpha"/>
        <constructor-arg ref="address"/>
        <constructor-arg ref="department"/>
    </bean>

</beans>
```

The **address** and **department** beans are simple: all their values are plain text, so they use `value`. The **employee** bean mixes both: the first two arguments (`101`, `Alpha`) are plain values, and the last two use `ref` to point to the beans we already defined.

> **Order matters inside a bean.** The `<constructor-arg>` tags are matched to the constructor parameters in the order they are written: **id, name, address, department**. The order of the *beans* in the XML does not matter. Spring sees that **employee** depends on **address** and **department**, and creates those two first.

Step 5: ref vs value

Every `<constructor-arg>` (and `<property>`) needs to know *what* to inject. That is decided by one attribute:

Attribute	Use it when	Example in this project
`value`	The thing to inject is a simple value: a number, a String, a boolean. Spring converts the text to the parameter type (for example, `"101"` becomes an `int`).	`value="Alpha"` `value="101"`
`ref`	The thing to inject is another object that is already a bean in the container. The text is the **id** of that bean.	`ref="address"` `ref="department"`

A quick way to remember it: if in Java you would write a **literal** (`101`, `"Alpha"`), use `value`. If you would pass a **variable that holds an object** (`address`, `department`), use `ref`.

> **ref takes the bean id, not the class name.** `ref="address"` works because the bean is declared with `id="address"`. If the id does not match any bean, Spring fails at startup with a `NoSuchBeanDefinitionException`. Also, `value="address"` would not work here: Spring would try to put the plain text `"address"` into an `Address` parameter, which is not possible.

Step 6: Create the IOC container and use the bean

In **App.java** we only request the **employee** bean. We never ask for **address** or **department**: Spring has already created them and injected them into the employee.

```
package com.amazon.employee;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class App {
    public static void main( String[] args )
    {
        String configFile = "beans.xml";

        ApplicationContext context = new ClassPathXmlApplicationContext(configFile);

        Employee employee1 = context.getBean("employee", Employee.class);

        employee1.displayEmployeeInfo();
    }
}
```

When we run the program, we get:

```
Employee Details:
Name: Alpha
Address:
    city = BLR
    state = Karnataka
    country=India
Department: engineering
```

What Spring does behind the scenes

1 Reads **beans.xml** and finds the three bean definitions.

2 Creates **address** and **department** using their all-args constructors and the `value` arguments.

3 Creates **employee**, passing the ready **address** and **department** objects into its constructor for the two `ref` arguments. `getBean()` then returns this employee.